Table of Content

Ansible

1. ¿Qué es Ansible?

Ansible es un motor de **automatización** de código abierto, potente y fácil de usar, que ayuda con la gestión de configuraciones, el despliegue de aplicaciones y la automatización de tareas. Su filosofía principal es ser minimalista y fiable a través de decisiones de diseño clave.

Herramienta de Provisión

Actúa principalmente como una herramienta de aprovisionamiento para configurar la infraestructura. Se puede usar para configurar servidores, desplegar aplicaciones y orquestar tareas en muchas máquinas simultáneamente, incluyendo infraestructura en la nube y redes.

SSH

Utiliza SSH (*Secure Shell*) como capa de transporte. Los hosts gestionados solo necesitan un servidor SSH, lo que evita instalar agentes de software adicionales. Esta arquitectura *sin agentes* reduce la complejidad de gestión.

Idempotente

Es fundamentalmente *idempotente*. Esto significa que puedes ejecutar el mismo script (*Playbook*) varias veces y el estado final será el mismo, sin efectos secundarios innecesarios. Por ejemplo, si un paquete ya está instalado, Ansible no intentará instalarlo de nuevo.

2. Instalación

La instalación es sencilla ya que es una herramienta basada en Python. Solo es necesario instalarlo en el *nodo de control*.

```
apt install ansible
```

En sistemas Debian y Ubuntu, es la forma más fácil usando el gestor de paquetes *APT*.

Ansible recomienda usar el gestor de paquetes de Python *pipx*.

```
apt install pipx
```

Instalamos Ansible:

```
pipx install --include-deps ansible
```

Útil para obtener la última versión o instalarlo en entornos virtuales de Python específicos.

3. Beneficios

Sintaxis fácil de leer (YAML)

Utiliza YAML, un estándar amigable que permite que los Playbooks sean fáciles de escribir y entender.

Sin instalación en hosts remotos

Solo requieren un demonio SSH y Python (2.5 o superior).

Basado en inserción (Push)

El nodo de control envía las configuraciones a los hosts remotos, lo que permite un control inmediato.

Escalabilidad

Funciona igual de bien para un solo servidor que para grandes infraestructuras.

Módulos integrados

Ofrece cientos de módulos integrados para gestionar paquetes, servicios y archivos, abstrayendo la complejidad de los comandos del sistema.

Ansible Galaxy

El centro oficial para compartir y reutilizar roles de automatización de la comunidad.

Capa de abstracción delgada

Sus comandos se corresponden estrechamente con las acciones reales de administración, facilitando la resolución de problemas.

4. Vagrant y Terraform

Entender la diferencia entre Vagrant y Terraform es fundamental porque, aunque ambos son creados por HashiCorp y comparten una filosofía de "Infraestructura como Código" (IaC), tienen propósitos y alcances muy distintos.

Vagrant está diseñado para crear y configurar **entornos de desarrollo reproducibles en tu propia computadora**. Levanta máquinas virtuales (VM) usando proveedores como VirtualBox o VMware. Un uso típico es cuando un desarrollador necesita un entorno Linux específico con bases de datos y librerías instaladas dentro de su Windows o Mac.

Terraform es una **herramienta de orquestación de infraestructura a gran escala**. No le importa tanto lo que pasa dentro de una sola VM, sino cómo se conectan cientos de recursos en la nube. Define y gestiona redes, instancias, bases de datos gestionadas (como RDS), buckets de almacenamiento y balanceadores de carga. Un ejemplo típico sería desplegar una arquitectura completa en AWS, Azure o Google Cloud de forma automatizada.

Ambos pueden usar herramientas de provisionamiento como Ansible, Chef o Puppet para instalar software. Normalmente Vagrant suele lanzar los scripts en cuanto arranca la máquina en local, mientras que Terraform deja la infraestructura creada para luego usar otras herramientas.

5. Ansible y Vagrant

Vagrant crea y gestiona máquinas virtuales, mientras que Ansible se utiliza como el *provisionador* para realizar las configuraciones específicas dentro de esas máquinas.

Vagrant crea la MV

Vagrant gestiona la configuración inicial, es decir, crea y configura la máquina virtual basándose en un *Vagrantfile*. Esto incluye seleccionar el sistema operativo (*box*) y configurar las direcciones de red.

Ansible hace la provisión

Una vez que la MV está ejecutándose, Ansible se usa como un *provisionador* para realizar las configuraciones específicas *dentro* de la MV. En este paso es donde instalaría el software necesario, configuras los servicios, y desplieguas una apliación, asegurándose de que el estado interno de la MV se adapte a la especificación deseada.

5.1. Ejemplo de Configuración del Proyecto

Para ilustrar la integración, una configuración típica de proyecto implica inicializar un entorno Vagrant y ponerlo en línea.

5.1.1. Configuración de Vagrant

```
mkdir web; cd web
```

Crean y acceden a un nuevo directorio de proyecto.

```
vagrant init -m debian/bookworm64
```

Crea un archivo `Vagrantfile` básico utilizando la imagen Debian especificada. La opción *-m* crea un archivo mínimo.

```
vagrant up
```

Lee el archivo `Vagrantfile`, descarga la imagen (si es necesario) y configura la máquina virtual.

5.1.2. Comprobar Conexión SSH

Para usar Ansible, debes saber cómo conectarte al host remoto (la máquina virtual de Vagrant). Vagrant facilita el acceso a esta información.

```
vagrant ssh-config
```

Este comando muestra los detalles específicos de la conexión SSH de la máquina virtual Vagrant en ejecución. Esto es fundamental, ya que Vagrant suele usar un puerto asignado aleatoriamente para su conexión SSH, y la clave privada la gestiona internamente.

Listing 1: Datos relevantes de conexión

```
Nombre de host: 127.0.0.1
Usuario: vagrant
Puerto: 2222
Archivo de identidad: /home/fraya/Work/Ansible/web/.vagrant/machines/default/virtualbox/private_key
```

La salida proporciona la dirección IP (*HostName*), el nombre de usuario (*User*), el puerto dinámico y la ubicación del archivo de clave privada SSH (*IdentityFile*) necesario en el nodo de control. Estos son los parámetros exactos que Ansible necesita para conectarse y autenticarse.

5.1.3. Inventario para Vagrant

Se define un archivo `hosts` con los parámetros de conexión:

```
web ansible_ssh_host=127.0.0.1 ansible_ssh_port=2222 ansible_ssh_user=vagrant ansible_ssh_private_key_file=.vagrant/machines/default/virtualbox/private_key
```

Esto define un único host llamado *web*. Las variables subsiguientes (con el prefijo $ansible_{ssh\_}$) son parámetros de conexión que indican a Ansible cómo acceder al host: la IP, el puerto, el usuario y la ruta de la clave privada. Este inventario creado manualmente es la forma más sencilla de establecer conectividad con la máquina virtual.

5.1.4. Integración de aprovisionamiento

Para evitar el uso de comandos manuales ad hoc, el `Vagrantfile` se puede configurar para ejecutar automáticamente Playbooks de Ansible durante el proceso *vagrant up*.

Listing 2: Fichero Vagrantfile

```
 1: Vagrant.configure("2") do |cfg|
 2:   cfg.vm.box = "debian/bookworm64"
 3:   cfg.ssh.insert_key = false
 4:   cfg.vm.define "web" do |web|
 5:     web.vm.hostname = "web.test"
 6:     web.vm.network :private_network, ip: "192.168.57.10"
 7:     web.vm.provision :ansible do |ansible|
 8:       ansible.playbook = "web-playbook.yml"
 9:     end
10:   end # web
11: end # Vagrant
```

Línea 2

Establece la imagen base y desactiva la inserción de claves. Esto evita que Vagrant reemplace la clave predeterminada no segura, lo cual es útil para las conexiones de Ansible en modo de desarrollo.

Línea 4

Define una máquina llamada *web*, estableciendo su nombre de host y una dirección IP privada estática (*192.168.57.10*).

Línea 7

Incluye un bloque *provision :ansible*. Esto le indica a Vagrant que use Ansible para configurar la máquina virtual.

Línea 8

Fundamentalmente, *ansible.playbook = "web-playbook.yml* especifica el archivo de playbook de Ansible que se ejecutará en la máquina virtual después de su arranque.

6. Ansible y Terraform

Terraform (el orquestador)

se encarga de la Infraestructura como Código (IaC). Su trabajo es hablar con los proveedores de nube (AWS, Azure, GCP) para crear los componentes básicos. Crea las redes, subredes, firewalls, instancias de servidores (EC2), bases de datos (RDS) y balanceadores de carga.

Ansible hace la provisión

Una vez que la MV está ejecutándose, Ansible se usa como un *provisionador* para realizar las configuraciones específicas *dentro* de la MV. En este paso es donde instalaría el software necesario, configuras los servicios, y desplieguas una apliación, asegurándose de que el estado interno de la MV se adapte a la especificación deseada.

Terraform se encargará de pasar la dirección IP de las MV creadas y Ansible ejecutará las acciones para aprovisionarla.

7. Inventario de Ansible con Terraform

El **Inventario** de Ansible define los hosts que Ansible gestionará. Para una conexión ad-hoc, los detalles de conexión obtenidos se colocan en un archivo de inventario simple, que suele llamarse `hosts`.

7.1. Inventario básico

Imagina que Terraform acaba de crear dos servidores web y una base de datos. Terraform te entrega las IPs a través de sus outputs. Tu archivo de inventario de Ansible (*hosts.ini*) se vería así:

Listing 3: Fichero hosts.init

```
[web_servers]
web01 ansible_host=34.201.10.45
web02 ansible_host=34.201.10.46

[db_servers]
db01  ansible_host=10.0.1.15

[all:vars]
ansible_user=admin
ansible_ssh_private_key_file=~/.ssh/id_rsa
```

7.2. Inventario dinámico

En lugar de copiar y pegar IPs a mano, Ansible tiene plugins de inventario que consultan directamente a la nube (AWS, Azure, etc.) para ver que creó Terraform basándose en *etiquetas (Tags)*. Esto está más allá de la intención del tutorial, pero a continuación pongo un ejemplo.

Listing 4: Fichero main.tf

```
resource "aws_instance" "web" {
  ami           = "ami-0c55b31ad2c455b55"
  instance_type = "t2.micro"
  key_name      = "your-ssh-key-pair"

  tags = {
    Name = "WebServer"
    Role = "web_server"
  }
}
```

En Ansible utilizando el plugin de AWS podemos buscar instancias que tengan el *tag Role igual a* web_{server}.

7.3. Terraform genera el inventario

Una técnica muy usada para proyectos pequeños es hacer que Terraform escriba el archivo de Ansible automáticamente usando el recurso $local_{file}$.

Listing 5: Fichero main.tf

```
resource "local_file" "ansible_inventory" {
  content  = <<EOT
[web_servers]
${aws_instance.servidor_web.public_ip}

[db_servers]
${aws_instance.db_instance.private_ip}
EOT
  filename = "inventory.ini"
}
```

Esto creará un fichero *inventory.ini* con los datos.

```
ansible-playbook -i inventory.ini site.yml
```

8. Prueba (Comando Ad-hoc)

Una prueba rápida utilizando el módulo `ping` integrado es la mejor manera de verificar que Ansible puede conectarse y autenticarse con el host remoto utilizando el archivo de inventario.

```
$ ansible web -i hosts -m ping
```

Este comando indica a Ansible que se dirija al host *web*, utilice el archivo de inventario *-i hosts* y ejecute el módulo integrado *-m ping*. Una respuesta exitosa (por ejemplo, `SUCCESS => {"changed": false, "ping": "pong"}`) confirma que la conexión SSH de Ansible, la autenticación y la ejecución remota de Python funcionan correctamente.

9. Personalización

9.1. Simplificación con `ansible.cfg`

Se puede crear un archivo `ansible.cfg` para centralizar la ruta del inventario, el usuario remoto y la configuración de las llaves, evitando repetir flags en la línea de comandos.

9.2. Comandos Ad-hoc

Módulo command

Para ejecutar comandos simples como *uptime*.

Privilegios (become)

El flag *-b* permite ejecutar tareas como root (usando *sudo*).

Gestión de paquetes

Usar el *módulo apt* para asegurar que programas como Nginx estén instalados.

9.3. Inventario en formato YAML

El archivo `inventory.yml` permite organizar hosts en grupos y asignar variables comunes. A continuación te muestro el mismo inventario que aparecía antes en formato `.ini` en formato `.yaml`.

Listing 6: Fichero ~inventory.yml

```
all:
vars:
  ansible_user: admin
  ansible_ssh_private_key_file: ~/.ssh/id_rsa
children:
  web_servers:
    hosts:
      web01:
        ansible_host: 34.201.10.45
      web02:
        ansible_host: 34.201.10.46
  db_servers:
    hosts:
      db01:
        ansible_host: 10.0.1.15
```

10. Playbooks

Los **Playbooks** son la parte más importante de Ansible; definen tareas secuenciales en archivos YAML para lograr una configuración repetible.

```
$ ansible-playbook -i inventory.yml web-playbook.yml
```

10.1. Características principales

Idempotencia

Puedes ejecutar el mismo playbook varias veces y el resultado será el mismo, sin realizar cambios si el sistema ya está en el estado deseado.

Formato YAML

Utiliza una sintaxis legible para humanos.

Orden de ejecución

Las tareas se ejecutan de arriba hacia abajo.

10.2. Componentes de un Playbook

Un Playbook es esencialmente una lista de acciones. Cada acción mapea un grupo de hosts con tareas específicas.

```
name: Configurar servidor web Apache
hosts: web_servers
become: yes  # Ejecutar como sudo
vars:
  http_port: 80

tasks:

  - name: Asegurar que Apache esté instalado
    ansible.builtin.apt:
      name: apache2
      state: present

  - name: Iniciar el servicio de Apache
    ansible.builtin.service:

  - name: apache2
    state: started
    enabled: yes
```

Hosts y Usuarios

Define en qué máquinas se ejecutará el play (basado en tu inventario) y con qué privilegios.

Variables (vars)

Permiten parametrizar el playbook para que sea reutilizable en diferentes entornos (dev, prod).

Tareas (tasks)

Es el corazón del playbook. Cada tarea utiliza un módulo de Ansible (como apt, copy, service o shell).

Handlers

Son tareas especiales que solo se ejecutan si una tarea previa notifica un cambio (por ejemplo, reiniciar un servicio solo si el archivo de configuración cambió).

11. Estructura de un proyecto

Cuando los playbooks se vuelven complejos, se recomienda usar Roles, que permiten organizar las tareas, variables y archivos en una jerarquía de directorios estandarizada:

```
site.yml
webservers.yml
dbservers.yml
roles/
common/
webtier/
dbtier/
```

Para estructurar proyectos más reales (grandes) consulta **Ansible sample setup**.

12. Roles

Los Roles son la forma evolucionada de organizar contenido en Ansible. Mientras que un Playbook básico es un archivo único (que puede volverse enorme y difícil de leer), los roles permiten desglosar el código en una estructura de directorios estandarizada y reutilizable.

Imagina un rol como un "paquete" autónomo que contiene todo lo necesario para realizar una función específica (instalar una base de datos, configurar un firewall, etc.).

12.1. Estructura de Directorios de un Rol

La jerarquía de un rol (por ejemplo, un rol llamado apache) se vería así:

```
roles/
apache/
├── tasks/           # Tareas principales (main.yml)
├── handlers/        # Manejadores, como reinicios de servicios
├── templates/       # Archivos .j2 (Jinja2) con variables
├── files/           # Archivos estáticos para copiar
├── vars/            # Variables de alta prioridad
├── defaults/        # Variables por defecto (baja prioridad)
├── meta/            # Dependencias y autoría
└── README.md        # Documentación del rol
```

12.2. Por qué usar Roles?

Reutilización

Puedes usar el mismo rol de nginx en diez proyectos distintos sin copiar y pegar código.

Mantenibilidad

Si necesitas cambiar la configuración de la base de datos, sabes exactamente que debes ir a roles/db/tasks/main.yml.

Orden

El playbook principal (site.yml) se vuelve extremadamente limpio, sirviendo solo para llamar a los distintos roles.

12.3. Ejemplo de uso de roles en un Playbook

```
hosts: web_servers
roles:
  - common  # Configuración básica (usuarios, updates)
  - apache  # Instalación y config de web server
```

13. Ansible Galaxy

Los roles son tan potentes que existe una plataforma llamada **Ansible Galaxy**. Es como el "App Store" o el "npm" de Ansible. Puedes descargar roles creados por la comunidad (o por empresas como Red Hat, Nginx o Docker) e integrarlos en tu infraestructura inmediatamente.

```
$ ansible-galaxy install geerlingguy.apache
```

14. Referencias

- https://developer.hashicorp.com/validated-patterns/terraform/terraform-integrate-ansible-automation-platform
- https://scalr.com/learning-center/ultimate-guide-to-using-terraform-with-ansible/
- https://docs.ansible.com/projects/ansible/latest/installation_guide/intro_installation.html
- https://docs.ansible.com/projects/ansible/latest/tips_tricks/sample_setup.html
- https://galaxy.ansible.com/ui/

Author: Fernando Raya
Created: 2026-05-06 mié 15:35
Validate